



Chapter 8

Time and Space Analysis

อัลกอริทึม (Algorithm)

- ขั้นตอนวิธีที่ใช้ในการแก้ปัญหา (หาเส้นทางจาก Initial state -> End state)
- อัลกอริทึม เป็นลักษณะของแนวคิด ซึ่งสามารถแสดงออกมาได้หลายรูปแบบ
 - ซูโดโค้ด (Pseudo code)
 - แผนผัง (Flowchart)
 - โปรแกรม (Program)

ซูโดโค้ด (Pseudo Code)

Algorithm sample (pageNumber)

This algorithm reads a file and prints a report.

Pre pageNumber passed by reference

Post Report Printed

 pageNumber contains number of pages in report

Return Number of lines printed

1 loop (not end of file)

1 read file

2 if (full page)

 1 increment page number

 2 write page heading

3 end if

4 write report line

5 increment line count

2 end loop

3 return line count

end sample

การวิเคราะห์อัลกอริทึม

(Algorithm Analysis)

- ปัญหาหนึ่งๆ สามารถถูกแก้ได้ด้วยหลากหลายวิธี (หลายอัลกอริทึม)
- เพื่อเลือกวิธีที่ดี เราต้องสามารถเปรียบเทียบประสิทธิภาพของอัลกอริทึมได้
- การวิเคราะห์อัลกอริทึม พิจารณา
 - Time Complexity : เวลาที่ใช้ในการรันโปรแกรม
 - Space Complexity : พื้นที่หรือหน่วยความจำที่ใช้ในการรันโปรแกรม

ประสิทธิภาพของอัลกอริทึม (Algorithm efficiency)

- พิจารณาประสิทธิภาพจากจำนวนของอิลิเมนต์ที่ถูกโพรเซส
- $f(n) = \text{efficiency}$
- ถ้าฟังก์ชันเป็น Linear (no loops)
 - Efficiency = จำนวนคำสั่งที่ทำงานทั้งหมด

Linear Loops

- มีการเพิ่ม/ลดค่าภายในลูปอย่างคงที่

```
for (i=0; i<1000; i++)
```

```
    application code
```

```
end for
```

Iterations : 1000 ($f(n) = 1000$)

```
for (i=0; i<1000; i+=2)
```

```
    application code
```

```
end for
```

Iterations : 500

```
for (i=0; i<n; i++)
```

```
    application code
```

```
end for
```

; $f(n) = n$

```
for (i=0; i<n; i+=2)
```

```
    application code
```

```
end for
```

; $f(n) = \frac{n}{2}$

Logarithmic Loops

- เพิ่ม/ลดค่าภายในลูปเป็นจำนวนเท่าตัวด้วยการคูณหรือหาร

for (i=1; i<1000; i*=2)

application code

end for $\lceil \log_2 1000 \rceil$

for (i=1000; i>1; i/=2)

application code

end for

Iterations : $\lceil \log_2 1000 \rceil$

for (i=1; i<n; i*=2)

application code

end for $\log_2 n$

for (i=n; i>1; i/=2)

application code

end for

; $f(n) = 1 \cdot \log_2 n$

← มาจากว่า
ให้ดูที่
ไป

Logarithmic Loops (cont.)

Multiply		Divide	
Iteration	Value of i	Iteration	Value of i
1	1	1	1000
2	2	2	500
3	4	3	250
4	8	4	125
5	16	5	62
6	32	6	31
7	64	7	15
8	128	8	7
9	256	9	3
10	512	10	1
(exit)	1024	(exit)	0

Nested Loops

- เป็นลักษณะของลูปซ้อนลูป
- Iterations = outer loop iterations * inner loop iterations
- Linear Logarithmic

for (i=0; i<10; i++)

for (j=1; j<10; j*=2)

application code

end for

end for

Iterations : 40
 $10 \cdot \lceil \log_2 10 \rceil$

$10 \cdot (x)$

$\log_2 10$

$f(n) = n \cdot \lceil \log_2 n \rceil$

Nested Loops (cont.)

- Quadratic

```
for (i=0; i<10; i++)
```

```
    for (j=0; j<10; j++)
```

```
        application code
```

```
    end for
```

```
end for
```

◦ Iterations : 10×10 ; $f(n) = n^2$
 100

Nested Loops (cont.)

- Dependent Quadratic

for (i=1; i<=10; i++) 10

for (j=0; j<i; j++) 1

application code

end for

end for

Iterations : $1+2+3+4+\dots+10$; $f(n) = 1+2+\dots+n$
55 $= \frac{n}{2} (n+1)$

i	
1	1
2	2
3	3

Big-O Notation

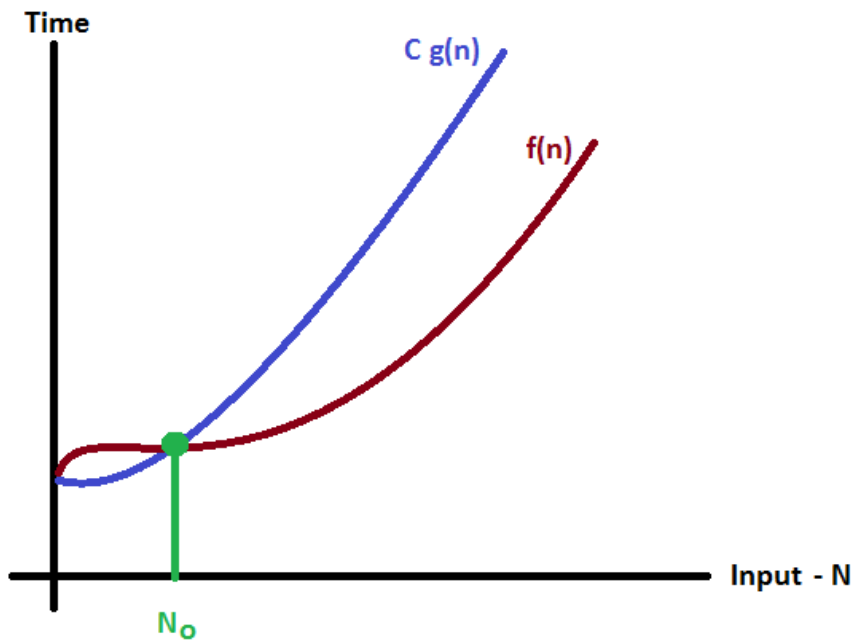
- การวัดประสิทธิภาพเน้นที่ข้อมูลปริมาณมาก และสนใจค่าประมาณ (Approximation) มากกว่า ด้วยฟังก์ชันบิกโอ (Big-O)
- ซึ่งเป็นการพิจารณาจาก อัตราการเติบโตของฟังก์ชัน -> ดูจากค่าที่มีผลกระทบมากที่สุดค่าเดียว

Big-O Notation

- $f(n) = O(g(n))$ เมื่อ

$$f(n) \leq cg(n), \quad \text{for } c > 0 \text{ และ } n \geq n_0 \geq 1$$

- $f(n)$ เป็นบีกโอของ $g(n)$



Big-O Notation

- พิจารณาแต่ละเทอมให้มีสัมประสิทธิ์เป็น 1
- สนใจเฉพาะเทอมที่ใหญ่ที่สุด
 - $\log n$ n $n \log n$ n^2 n^3 ... n^k 2^n $n!$

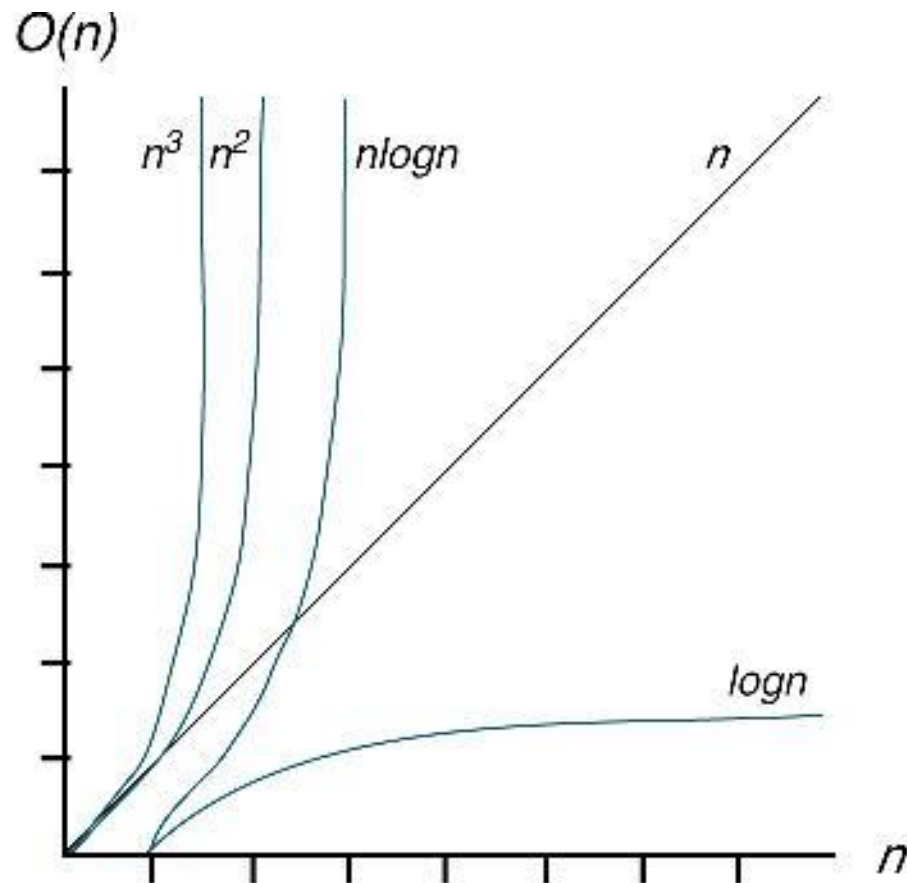
น้อย $\longrightarrow$ มาก
ในรูป big O ของเทอม

Standard Measures of Efficiency

Big O notation

N	N^2	N^3	2^N	$\log_2 N$	$N \log_2 N$
1	1	1	2	0	0
2	4	8	4	1	2
4	16	64	16	2	8
8	64	512	256	3	24
16	256	4096	65536	4	64
32	1024	32768	2147483648	5	160
64	4096	262144	?	6	384
128	16384	2097152	?	7	896
256	65536	16777216	?	8	2048

Standard Measures of Efficiency (cont.)



Standard Measures of Efficiency (cont.)

Efficiency	Big-O	Iterations	Estimated Time
Logarithmic	$O(\log n)$	14	microseconds
Linear	$O(n)$	10,000	seconds
Linear logarithmic	$O(n(\log n))$	140,000	seconds
Quadratic	$O(n^2)$	$10,000^2$	minutes
Polynomial	$O(n^k)$	$10,000^k$	hours
Exponential	$O(c^n)$	$2^{10,000}$	intractable
Factorial	$O(n!)$	$10,000!$	intractable

Measures of Efficiency for $n = 10,000$

Example

- $f(n) = 2n + 7$

- $f(n) = O(n)$

- $f(n) = n^3 + 3n - 8$

- $f(n) = O(n^3)$

$n^3 + n$

- $f(n) = 1, f(n) = 100$

- $f(n) = O(1)$

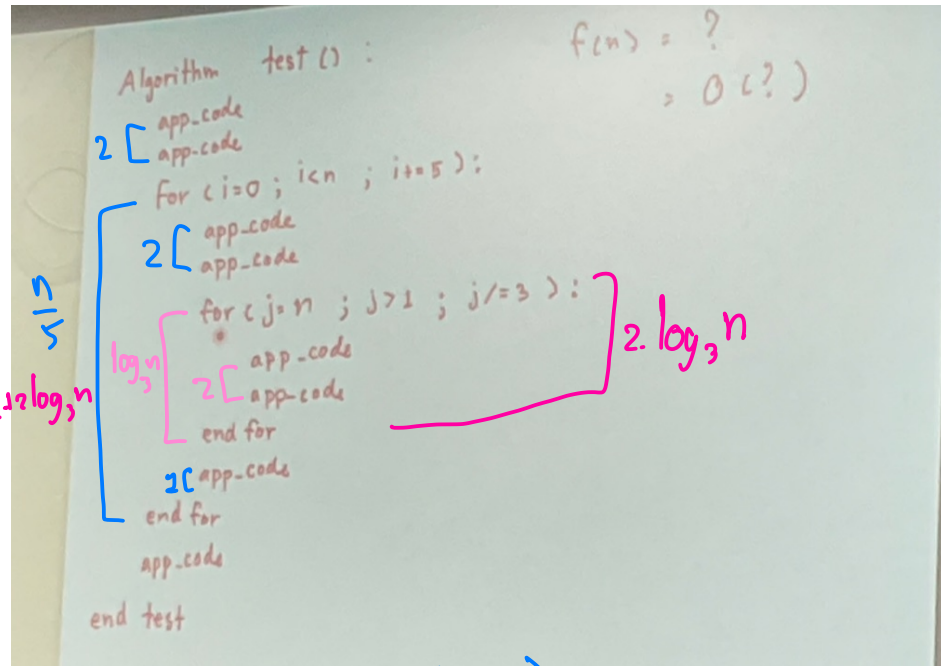
ไม่จำเป็นต้องขนาดข้อมูล n

Example

- $f(n) = 2n+7$
 - $f(n) = O(n)$

- $f(n) = n^3+3n-8$
 - $f(n) = O(n^3)$

- $f(n) = 1, f(n) = 100$
 - $f(n) = O(1)$



$$\begin{aligned}
 f(n) &= 3 + \frac{n}{5} (3 + 2 \log_3 n) \\
 &= 3 + \frac{3n}{5} + \frac{2n \log_3 n}{5} \\
 &= \frac{2n \log_3 n + 3n + 15}{5}
 \end{aligned}$$

$$O(n \log_3 n)$$

Big-O Analysis Example

- Add Square Matrices

4	2	1
0	-3	4
5	6	2

 +

6	1	7
3	2	-1
4	6	2

 =

10	3	8
3	-1	3
9	12	4

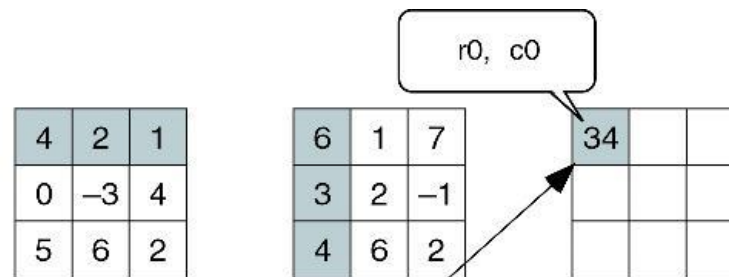
Big-O Analysis Example (cont.)

- Add Square Matrices

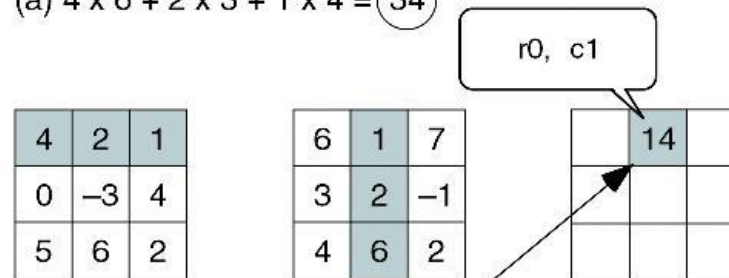
```
Algorithm addMatrix (matrix1, matrix2, size, matrix3)
Add matrix1 to matrix2 and place results in matrix3
  Pre  matrix1 and matrix2 have data
       size is number of columns or rows in matrix
  Post matrices added--result in matrix3
1 loop (not end of row)
  1 loop (not end of column)
    1 add matrix1 and matrix2 cells
    2 store sum in matrix3
  2 end loop
2 end loop
end addMatrix
```

Big-O Analysis Example (cont.)

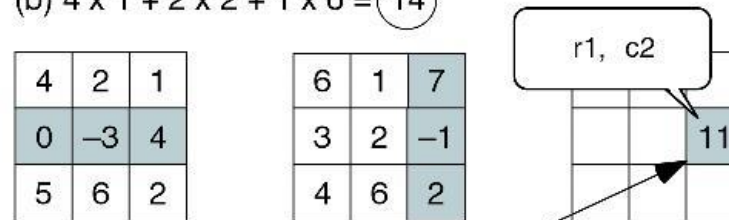
- Multiply Square Matrices



(a) $4 \times 6 + 2 \times 3 + 1 \times 4 = 34$



(b) $4 \times 1 + 2 \times 2 + 1 \times 6 = 14$



(c) $0 \times 7 + (-3) \times (-1) + 4 \times 2 = 11$

Big-O Analysis Example (cont.)

- Multiply Square Matrices

```
Algorithm multiMatrix (matrix1, matrix2, size, matrix3)
Multiply matrix1 by matrix2 and place product in matrix3
  Pre  matrix1 and matrix2 have data
       size is number of columns and rows in matrix
  Post matrices multiplied--result in matrix3
1 loop (not end of row)
  1 loop (not end of column)
    1 loop (size of row times)
      1 calculate sum of
        (all row cells) * (all column cells)
      2 store sum in matrix3
    2 end loop
  2 end loop
3 return
end multiMatrix
```

Quiz 1

- จงแสดง $f(n)$ และอัตราการเติบโตของโปรแกรมต่อไปนี้

Algorithm test1(n)

 for (i=0; i<n; i++)

 for (j=0; j<5; j++)

 app_code

 app_code

 end for

 app_code

end for

end test1

Quiz 2

- จงแสดง $f(n)$ และอัตราการเติบโตของโปรแกรมต่อไปนี้

Algorithm test2(n)

 for (i=1; i<n; i*=2)

 for (j=0; j<n; j+=2)

 app_code

 end for

 app_code

 app_code

 end for

end test2